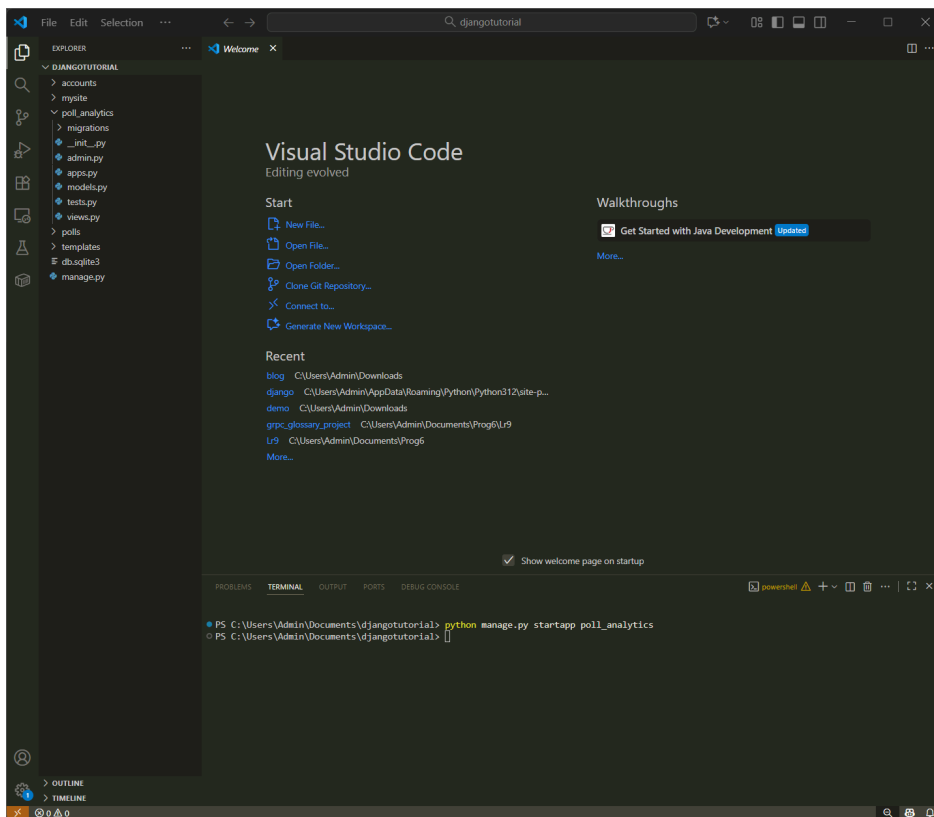


Лабораторная работа №6. Django REST Framework: (микро)сервисы

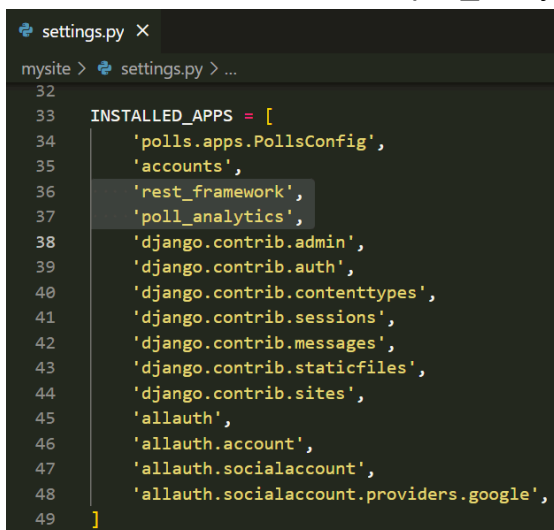
Статистика по голосованиям:

- общее количество голосований
- количество голосов на каждый вариант ответа
- процентное соотношение голосов

1. Создание нового приложения для аналитики



2. Подключение DRF и poll_analytics



3. Создание микросервиса статистики

Микросервис статистики по голосованию.

Возвращает количество голосов и процентное соотношение по каждому варианту.

```
views.py X
poll_analytics > views.py > QuestionStatsAPIView > get
1 from rest_framework.views import APIView
2 from rest_framework.response import Response
3 from rest_framework import status
4 from polls.models import Question, Choice
5 from django.db.models import Sum
6
7 class QuestionStatsAPIView(APIView):
8     def get(self, request, pk):
9         try:
10             question = Question.objects.get(pk=pk)
11         except Question.DoesNotExist:
12             return Response(
13                 {"error": "Question not found"},
14                 status=status.HTTP_404_NOT_FOUND
15             )
16
17         choices = question.choice_set.all()
18
19         total_votes = choices.aggregate(Sum('votes'))['votes__sum'] or 0
20
21         results = []
22         for choice in choices:
23             percent = (choice.votes / total_votes * 100) if total_votes > 0 else 0
24             results.append({
25                 "choice_text": choice.choice_text,
26                 "votes": choice.votes,
27                 "percentage": round(percent, 2)
28             })
29
30         data = {
31             "question_id": question.id,
32             "question_text": question.question_text,
33             "total_votes": total_votes,
34             "results": results
35         }
36
37         return Response(data)
```

4. Добавление маршрутов

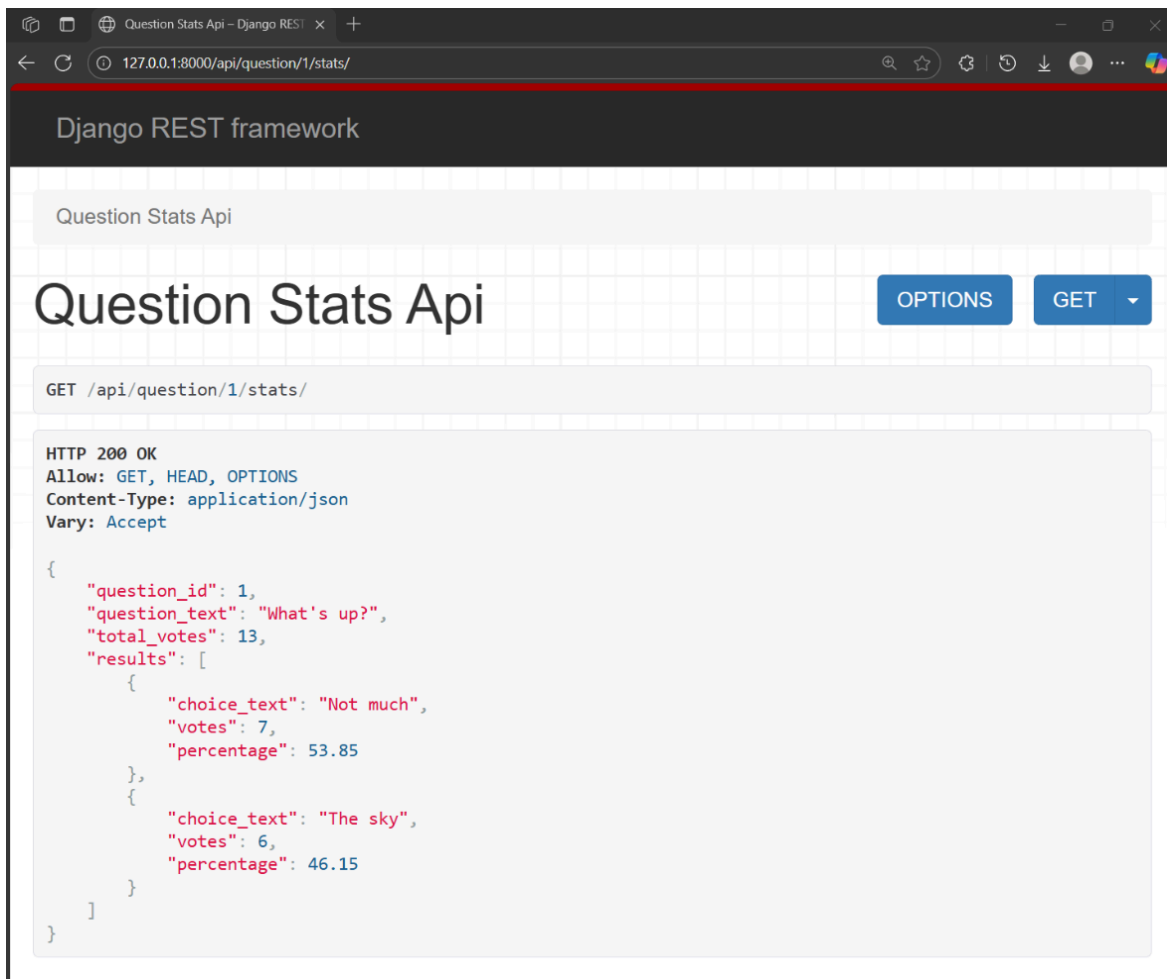
```
urls.py X
poll_analytics > urls.py > ...
1 from django.urls import path
2 from .views import QuestionStatsAPIView
3
4 urlpatterns = [
5     path('question/<int:pk>/stats/', QuestionStatsAPIView.as_view(), name='question-stats'),
6 ]
```

и основной [urls.py](#) проекта

```
urls.py  X
mysite > urls.py > ...
18 from django.urls import include, path
19
20 urlpatterns = [
21     path("polls/", include("polls.urls")),
22     path("admin/", admin.site.urls),
23     path("accounts/", include("allauth.urls")),
24     path("user/", include("accounts.urls")),
25     path('api/', include('poll_analytics.urls')),
26 ]
```

5. Результат:

<http://127.0.0.1:8000/api/question/1/stats/>



Микросервис:

- принимает ID вопроса (`/api/question/<id>/stats/`)
- извлекает все варианты (Choice) для этого вопроса
- считает общее количество голосов (`total_votes`)
- считает голоса и процент для каждого варианта
- возвращает JSON с полной статистикой

Сортировка и фильтрация данных

- дата проведения
- популярность
- количество голосов

6. Добавление нового Class во views.py
 микросервис для сортировки и фильтрации голосований.
 Поддерживает фильтрацию по дате и сортировку по количеству голосов или дате публикации.

```
views.py X
poll_analytics > views.py > QuestionFilterAPIView > get
8 class QuestionStatsAPIView(APIView):
38     return Response(data)
39
40 class QuestionFilterAPIView(APIView):
41     def get(self, request):
42         sort_by = request.GET.get('sort_by', 'date')
43         from_date = request.GET.get('from', None)
44         to_date = request.GET.get('to', None)
45
46         questions = Question.objects.all()
47
48         if from_date and to_date:
49             try:
50                 from_dt = timezone.datetime.strptime(from_date, '%Y-%m-%d')
51                 to_dt = timezone.datetime.strptime(to_date, '%Y-%m-%d')
52                 questions = questions.filter(pub_date__range=[from_dt, to_dt])
53             except ValueError:
54                 return Response(
55                     {"error": "Invalid date format. Use YYYY-MM-DD"},
56                     status=status.HTTP_400_BAD_REQUEST
57                 )
58
59         if sort_by in ('votes', 'popularity'):
60             questions = questions.annotate(total_votes=Sum('choice_votes')).order_by('-total_votes')
61         elif sort_by == 'date':
62             questions = questions.order_by('-pub_date')
63         else:
64             return Response(
65                 {"error": "Invalid sort_by parameter. Use 'date', 'votes' or 'popularity'"},
66                 status=status.HTTP_400_BAD_REQUEST
67             )
68
69         result = []
70         for q in questions:
71             total_votes = q.choice_set.aggregate(Sum('votes'))['votes__sum'] or 0
72             result.append({
73                 "id": q.id,
74                 "question_text": q.question_text,
75                 "pub_date": q.pub_date,
76                 "total_votes": total_votes
77             })
78
79     return Response({"questions": result})
```

7. Добавление маршрута в urls.py

```
urls.py X
poll_analytics > urls.py > ...
1 from django.urls import path
2 from .views import QuestionStatsAPIView, QuestionFilterAPIView
3
4 urlpatterns = [
5     path('question/<int:pk>/stats/', QuestionStatsAPIView.as_view(), name='question-stats'),
6     path('questions/filter/', QuestionFilterAPIView.as_view(), name='question-filter'),
7 ]
```

8. Результат:

Пример 1 - сортировка по дате:

The screenshot shows a web browser window with the URL `127.0.0.1:8000/api/questions/filter/?sort_by=date`. The page title is "Question Filter Api". The response is a JSON array of two question objects, sorted by date. The first object has an ID of 1 and a publication date of "2025-10-03T18:27:32Z" with 13 votes. The second object has an ID of 2 and a publication date of "2025-10-09T17:46:06.387783Z" with 0 votes.

```
HTTP 200 OK
Allow: GET, HEAD, OPTIONS
Content-Type: application/json
Vary: Accept

{
  "questions": [
    {
      "id": 1,
      "question_text": "What's up?",
      "pub_date": "2025-10-03T18:27:32Z",
      "total_votes": 13
    },
    {
      "id": 2,
      "question_text": "What's up?",
      "pub_date": "2025-10-09T17:46:06.387783Z",
      "total_votes": 0
    }
  ]
}
```

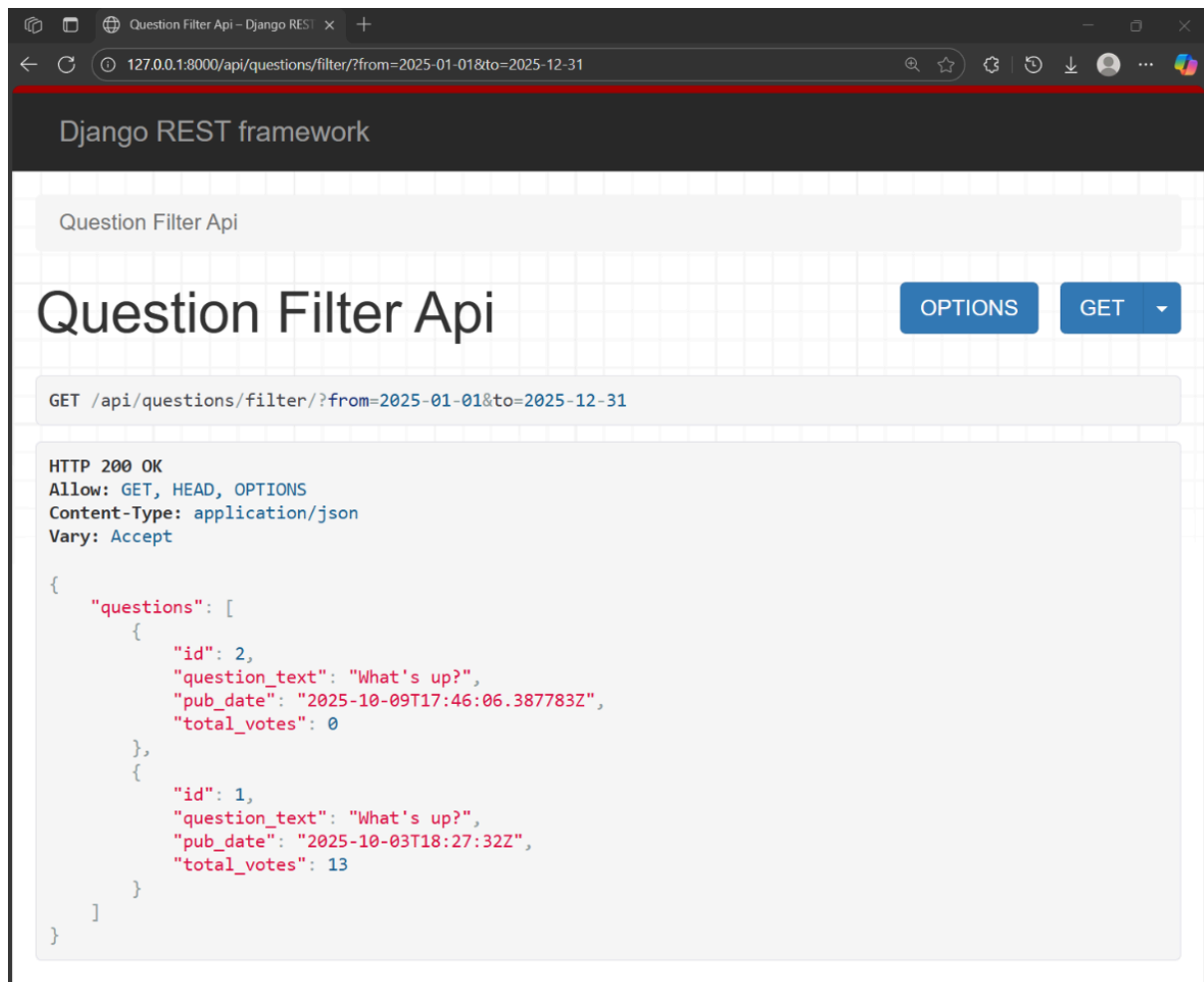
Пример 2 - сортировка по популярности:

The screenshot shows a web browser window with the URL `127.0.0.1:8000/api/questions/filter/?sort_by=votes`. The page title is "Question Filter Api". The response is a JSON array of two question objects, sorted by the number of votes. The first object has an ID of 1 and 13 votes, with a publication date of "2025-10-03T18:27:32Z". The second object has an ID of 2 and 0 votes, with a publication date of "2025-10-09T17:46:06.387783Z".

```
HTTP 200 OK
Allow: GET, HEAD, OPTIONS
Content-Type: application/json
Vary: Accept

{
  "questions": [
    {
      "id": 1,
      "question_text": "What's up?",
      "pub_date": "2025-10-03T18:27:32Z",
      "total_votes": 13
    },
    {
      "id": 2,
      "question_text": "What's up?",
      "pub_date": "2025-10-09T17:46:06.387783Z",
      "total_votes": 0
    }
  ]
}
```

Пример 3 - фильтрация по диапазону дат:



Графики и диаграммы

Микросервис, который строит столбчатую диаграмму (bar chart) для каждого голосования, показывая количество или процент голосов за каждый вариант ответа.

9. Добавление нового Class в views.py

```
8 import plotly.graph_objects as go
9 import plotly.io as pio
10 import base64
11 from io import BytesIO
```

Микросервис для генерации графика голосования (столбчатая диаграмма). Возвращает SVG прямо в JSON

```

views.py
poll_analytics > views.py > QuestionChartAPIView > get
45 class QuestionFilterAPIView(APIView):
46     def get(self, request):
84         return Response({"questions": result})
85
86 class QuestionChartAPIView(APIView):
87     def get(self, request, pk):
88         try:
89             question = Question.objects.get(pk=pk)
90         except Question.DoesNotExist:
91             return Response(
92                 {"error": "Question not found"},
93                 status=status.HTTP_404_NOT_FOUND
94             )
95
96         choices = question.choice_set.all()
97         total_votes = choices.aggregate(Sum('votes'))['votes__sum'] or 0
98
99         labels = [c.choice_text for c in choices]
100        votes = [c.votes for c in choices]
101        percentages = [
102            round((c.votes / total_votes * 100), 2) if total_votes > 0 else 0
103            for c in choices
104        ]
105
106        fig = go.Figure(data=[
107            go.Bar(x=labels, y=percentages, text=percentages, textposition='auto')
108        ])
109        fig.update_layout(
110            title=f"Voting results: {question.question_text}",
111            xaxis_title="Answer choices",
112            yaxis_title="Vote percentage (%)",
113            template="plotly_white"
114        )
115
116        svg_bytes = pio.to_image(fig, format="svg")
117        svg_str = svg_bytes.decode('utf-8')
118
119        return Response({
120            "question": question.question_text,
121            "total_votes": total_votes,
122            "labels": labels,
123            "percentages": percentages,
124            "chart_svg": svg_str
125        })

```

10. Добавление маршрута в urls.py

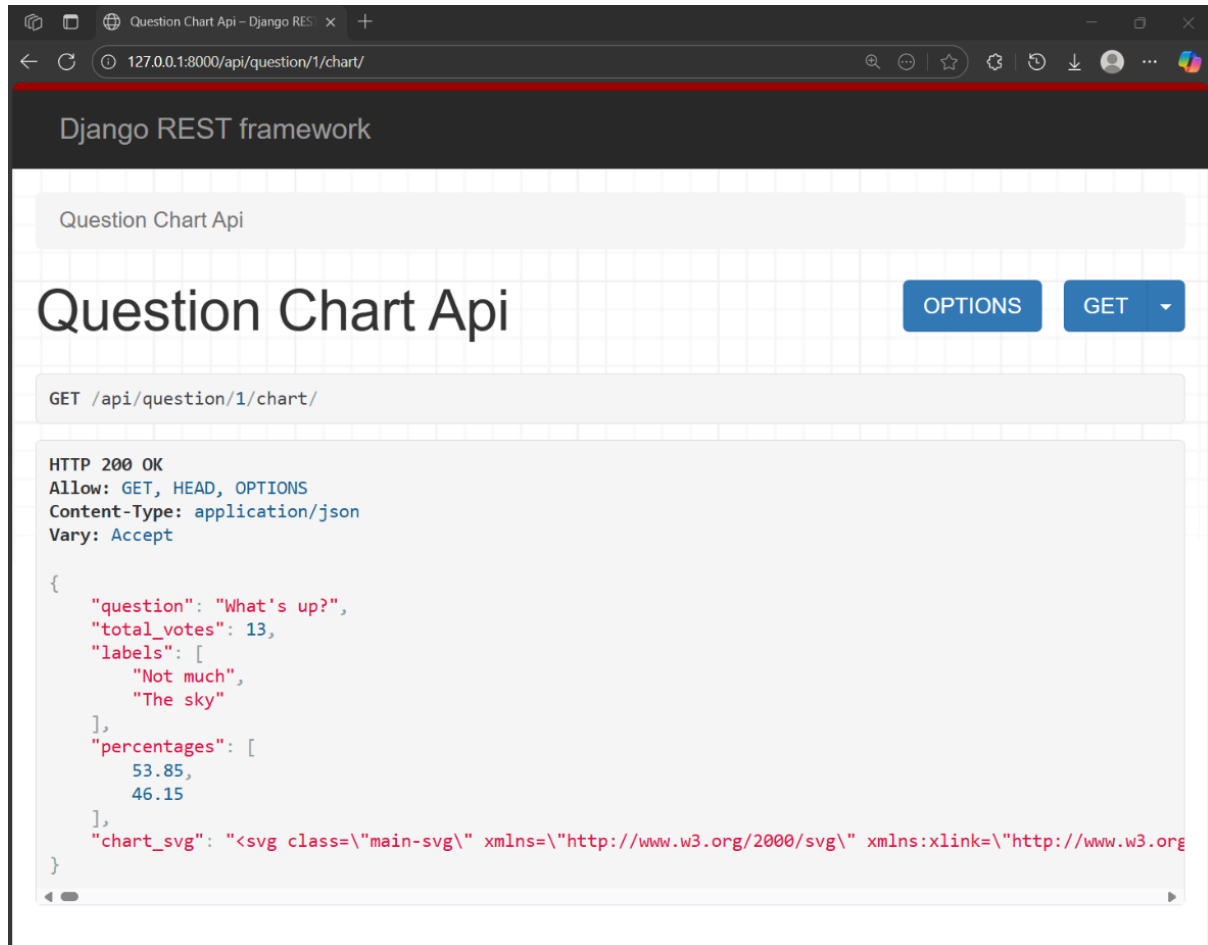
```

urls.py
poll_analytics > urls.py > ...
1 from django.urls import path
2 from .views import QuestionStatsAPIView, QuestionFilterAPIView, QuestionChartAPIView
3
4 urlpatterns = [
5     path('question/<int:pk>/stats/', QuestionStatsAPIView.as_view(), name='question-stats'),
6     path('questions/filter/', QuestionFilterAPIView.as_view(), name='question-filter'),
7     path('question/<int:pk>/chart/', QuestionChartAPIView.as_view(), name='question-chart'),
8 ]

```

11. Результат

<http://127.0.0.1:8000/api/question/1/chart/>



JSON с полем “chart_svg” - внутри SVG-график (длинная строка <svg...>)
Если вставить этот SVG в HTML через JS, появится столбчатая диаграмма.

Экспорт данных в форматах CSV и JSON

12. Добавление нового Class в views.py

Микросервис для экспорта данных о голосованиях.

Поддерживает форматы: JSON и CSV.

```
import csv
from django.http import HttpResponse
```



```

views.py X
poll_analytics > views.py > QuestionExportAPIView > get
89 class QuestionChartAPIView(APIView):
128     })
129
130 class QuestionExportAPIView(APIView):
131     def get(self, request):
132         export_format = request.GET.get('format', 'json').lower()
133         questions = Question.objects.all().prefetch_related('choice_set')
134
135         if export_format == 'json':
136             data = []
137             for q in questions:
138                 choices = [
139                     {"choice_text": c.choice_text, "votes": c.votes}
140                     for c in q.choice_set.all()
141                 ]
142                 data.append({
143                     "id": q.id,
144                     "question_text": q.question_text,
145                     "pub_date": q.pub_date,
146                     "choices": choices
147                 })
148             return Response({"questions": data})
149
150         elif export_format == 'csv':
151             response = HttpResponse(content_type='text/csv')
152             response['Content-Disposition'] = 'attachment; filename="polls_export.csv"'
153
154             writer = csv.writer(response)
155             writer.writerow(['Question ID', 'Question Text', 'Choice', 'Votes', 'Publication Date'])
156
157             for q in questions:
158                 for c in q.choice_set.all():
159                     writer.writerow([q.id, q.question_text, c.choice_text, c.votes, q.pub_date])
160
161             return response
162
163         else:
164             return Response(
165                 {"error": "Invalid format. Use ?format=json or ?format=csv"},
166                 status=status.HTTP_400_BAD_REQUEST
167             )

```

13. Добавление маршрута в urls.py

```

urls.py X
poll_analytics > urls.py > ...
1 from django.urls import path
2 from .views import (QuestionStatsAPIView, QuestionFilterAPIView, QuestionChartAPIView, QuestionExportAPIView)
3
4 urlpatterns = [
5     path('question/<int:pk>/stats/', QuestionStatsAPIView.as_view(), name='question-stats'),
6     path('questions/filter/', QuestionFilterAPIView.as_view(), name='question-filter'),
7     path('question/<int:pk>/chart/', QuestionChartAPIView.as_view(), name='question-chart'),
8     path('questions/export/', QuestionExportAPIView.as_view(), name='question-export'),
9 ]

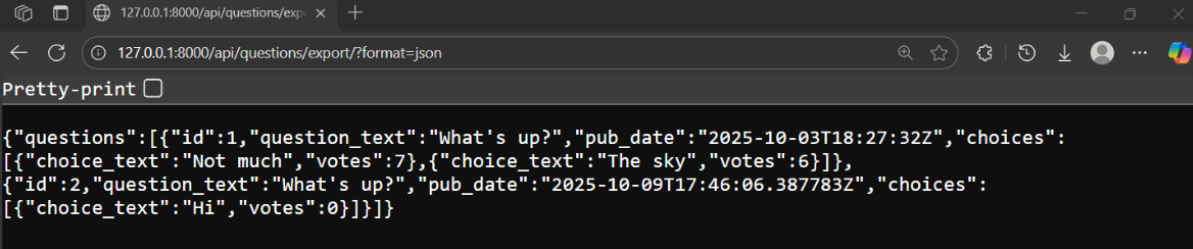
```

14. Результат

Пример 1 - экспорт в JSON:

<http://127.0.0.1:8000/api/questions/export/?format=json>

JSON с данными всех опросов:



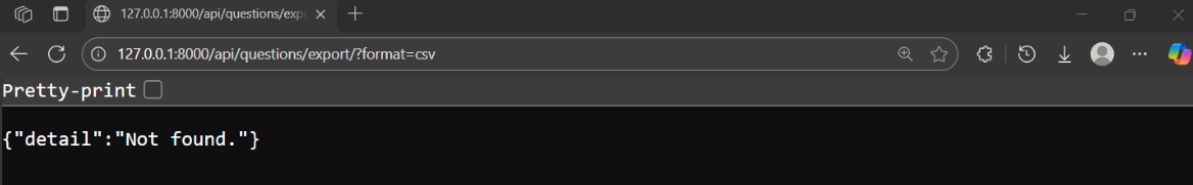
The screenshot shows a web browser window with the address bar displaying `127.0.0.1:8000/api/questions/export/?format=json`. Below the address bar, a 'Pretty-print' button is visible. The main content area displays a JSON object representing two survey questions. The first question has two choices: 'Not much' (7 votes) and 'The sky' (6 votes). The second question has one choice: 'Hi' (0 votes).

```
{
  "questions": [
    {
      "id": 1,
      "question_text": "What's up?",
      "pub_date": "2025-10-03T18:27:32Z",
      "choices": [
        {
          "choice_text": "Not much",
          "votes": 7
        },
        {
          "choice_text": "The sky",
          "votes": 6
        }
      ]
    },
    {
      "id": 2,
      "question_text": "What's up?",
      "pub_date": "2025-10-09T17:46:06.387783Z",
      "choices": [
        {
          "choice_text": "Hi",
          "votes": 0
        }
      ]
    }
  ]
}
```

Пример 2 - экспорт в CSV:

<http://127.0.0.1:8000/api/questions/export/?format=csv>

Браузер предложит скачать файл .csv, внутри которого строки



The screenshot shows a web browser window with the address bar displaying `127.0.0.1:8000/api/questions/export/?format=csv`. Below the address bar, a 'Pretty-print' button is visible. The main content area displays a JSON object with a 'detail' field set to 'Not found.', indicating that the CSV export endpoint is not implemented.

```
{
  "detail": "Not found."
}
```

Функционал страницы для поиска голосований

15. Создание шаблона страницы поиска search_polls.html

```
<> search_polls.html X
polls > templates > polls > <> search_polls.html > html > body > div.search-panel
1  {% load static %}
2  <!DOCTYPE html>
3  <html lang="en">
4  <head>
5      <meta charset="UTF-8">
6      <title>Search polls</title>
7      <script src="https://cdn.jsdelivr.net/npm/axios/dist/axios.min.js"></script>
8      <style>
9          body {
10              font-family: Arial, sans-serif;
11              margin: 30px;
12              display: flex;
13              gap: 20px;
14          }
15          .search-panel {
16              width: 30%;
17          }
18          .results {
19              flex: 1;
20          }
21          ul {
22              list-style: none;
23              padding: 0;
24          }
25          li {
26              padding: 8px;
27              margin: 5px 0;
28              background-color: #f5f5f5;
29              cursor: pointer;
30              border-radius: 4px;
31          }
32          li:hover {
33              background-color: #e0e0e0;
34          }
35          .chart-container {
36              margin-top: 20px;
37              border-top: 1px solid #ddd;
38              padding-top: 15px;
39          }
40      </style>
41  </head>
42  <body>
43
```

```

44 <div class="search-panel">
45   <h2>Search polls</h2>
46   <label>Sort by:</label>
47   <select id="sortSelect">
48     <option value="date">By date</option>
49     <option value="votes">By number of votes</option>
50   </select>
51
52   <h3>Date filter</h3>
53   <label>From:</label>
54   <input type="date" id="fromDate">
55   <label>To:</label>
56   <input type="date" id="toDate">
57   <button onclick="loadPolls()">Search</button>
58
59   <div style="margin-top: 15px;">
60     <h3>Export data</h3>
61     <button onclick="exportData('csv')">Download CSV</button>
62     <button onclick="exportData('json')">Download JSON</button>
63   </div>
64
65   <ul id="pollList"></ul>
66 </div>
67
68 <div class="results">
69   <h2 id="pollTitle">Select a poll</h2>
70   <div id="pollStats"></div>
71   <div id="pollChart" class="chart-container"></div>
72 </div>
73
74 <script>
75 async function loadPolls() {
76   const sortBy = document.getElementById("sortSelect").value;
77   const fromDate = document.getElementById("fromDate").value;
78   const toDate = document.getElementById("toDate").value;
79
80   let url = `/api/questions/filter/?sort_by=${sortBy}`;
81   if (fromDate && toDate) {
82     url += `&from=${fromDate}&to=${toDate}`;
83   }
84
85   try {
86     const response = await axios.get(url);

```

```

87     const list = document.getElementById("pollList");
88     list.innerHTML = "";
89
90     response.data.questions.forEach(q => {
91         const li = document.createElement("li");
92         li.textContent = `${q.question_text} (${q.total_votes} votes)`;
93         li.onclick = () => loadPollDetails(q.id, q.question_text);
94         list.appendChild(li);
95     });
96 } catch (error) {
97     console.error(error);
98     alert("Error loading polls");
99 }
100 }
101
102 async function loadPollDetails(questionId, questionText) {
103     document.getElementById("pollTitle").textContent = questionText;
104
105     try {
106         const statsRes = await axios.get(`/api/question/${questionId}/stats/`);
107         const statsDiv = document.getElementById("pollStats");
108         statsDiv.innerHTML = `
109             <h3>Statistics</h3>
110             <p>Total votes: ${statsRes.data.total_votes}</p>
111             <ul>
112                 ${statsRes.data.results.map(r => `<li>${r.choice_text}: ${r.votes} (${r.percentage}%)</li>`).join('')}
113             </ul>
114         `;
115     } catch (e) {
116         console.error(e);
117         alert("Error loading statistics");
118     }
119
120     try {
121         const chartRes = await axios.get(`/api/question/${questionId}/chart/`);
122         document.getElementById("pollChart").innerHTML = chartRes.data.chart_svg;
123     } catch (e) {
124         console.error(e);
125         alert("Error loading statistics");
126     }

```

```

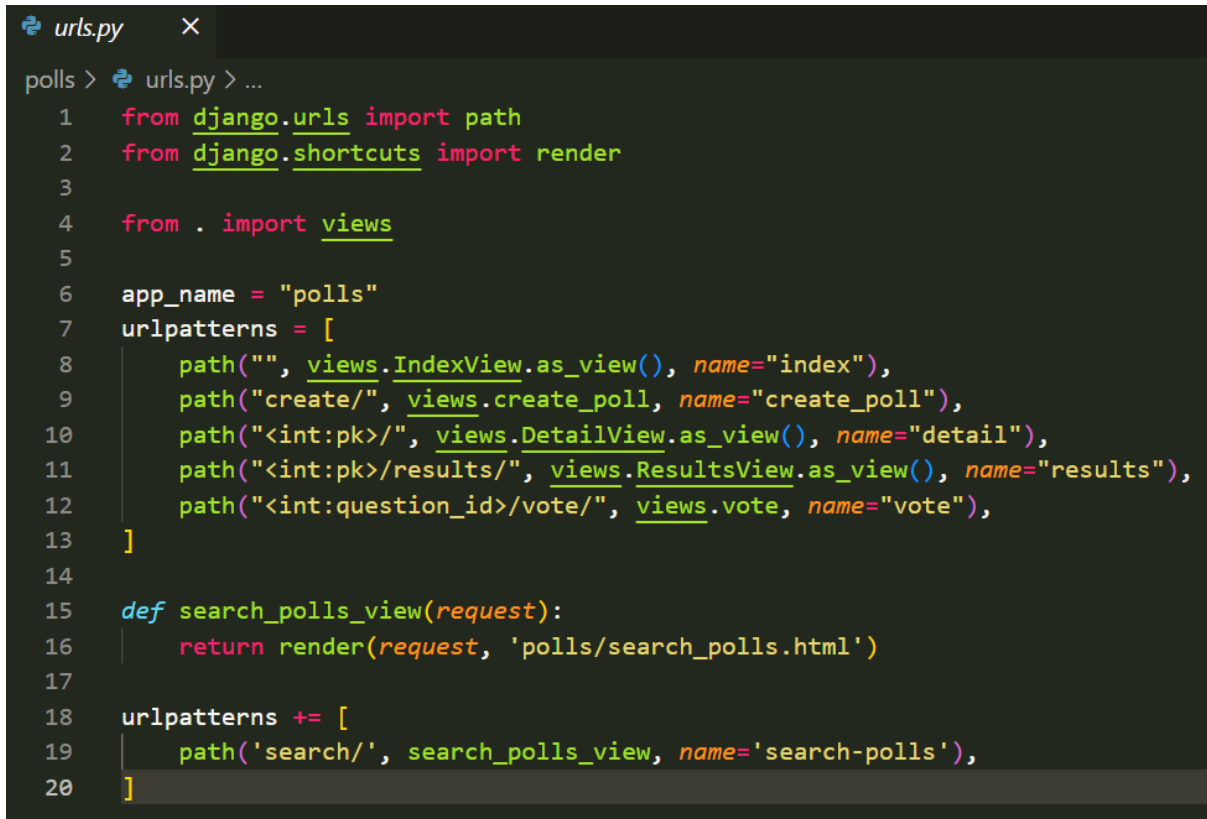
127     ]
128
129     async function exportData(format) {
130         const url = `/api/questions/export/?format=${format}`;
131         try {
132             if (format === 'csv') {
133                 const link = document.createElement('a');
134                 link.href = url;
135                 link.download = 'polls_export.csv';
136                 link.click();
137             } else {
138                 const res = await axios.get(url);
139                 const json = JSON.stringify(res.data, null, 2);
140                 const blob = new Blob([json], { type: 'application/json' });
141                 const link = document.createElement('a');
142                 link.href = URL.createObjectURL(blob);
143                 link.download = 'polls_export.json';
144                 link.click();
145             }
146         } catch (error) {
147             console.error(error);
148             alert("Error exporting data");
149         }
150     }
151
152     loadPolls();
153 </script>
154
155 </body>
156 </html>

```

Реализован функционал:

- фильтр по дате и сортировка по параметрам
- динамическая подгрузка списка опросов через Axios
- динамическая загрузка статистики
- диаграмма с процентами
- экспорт данных

16. Добавление маршрута для страницы в polls/urls.py



```
urls.py
polls > urls.py > ...
1  from django.urls import path
2  from django.shortcuts import render
3
4  from . import views
5
6  app_name = "polls"
7  urlpatterns = [
8      path("", views.IndexView.as_view(), name="index"),
9      path("create/", views.create_poll, name="create_poll"),
10     path("<int:pk>/", views.DetailView.as_view(), name="detail"),
11     path("<int:pk>/results/", views.ResultsView.as_view(), name="results"),
12     path("<int:question_id>/vote/", views.vote, name="vote"),
13 ]
14
15 def search_polls_view(request):
16     return render(request, 'polls/search_polls.html')
17
18 urlpatterns += [
19     path('search/', search_polls_view, name='search-polls'),
20 ]
```

17. Результат

<http://127.0.0.1:8000/polls/search/>

На странице:

- можно выбрать сортировку (по дате или голосам)
- при клике на голосование появится статистика и диаграмма справа
- экспорт результатов в CSV / JSON форматах

